

对抗性工程验收：怎样让“完成”脱离作者仍然成立

八仓归档中的独立扰动、事实分层与停止条件

——《保护零：从答案生成到事实成立》的执行方法篇

作者：然（NoctilumeDev）

文稿性质：论文体工程复盘·归档方法篇

成稿：2026 年 8 月

“一个永远只能产生 PASS 的体系，不是验收系统。”

体例说明

本文来自一次真实的八仓归档收尾。过程里既有产品缺陷，也有验收工具误判、宿主资源不足、GitHub 治理缺口和一次被及时叫停的过度施工。本文不把这段经历包装成预先设计好的实验，也不宣称形成了可以保证正确性的万能流程；它只对实际留下的固定坐标、失败现场、复验结果和停止裁决负责。

摘要

工程项目临近归档时，最容易出现一种温和却危险的错觉：源码能构建、测试通过、GitHub Actions 变绿，于是“完成”似乎已经成立。但作者、实现者、测试作者和审查者往往共享同一套上下文，也共享同一个错误前提；本地工作区、缓存、后台进程、浏览器地址、发布资产与远端规则还会继续影响结论。本文以八个公开仓库的归档收尾为纵向案例，提出一套对抗性工程验收方法：先冻结被验对象、合同、验证器与宿主前提，再以 Source、Build、Runtime、Browser、Failure、Artifact、Repository Governance、Cleanup 和 Public Readability 九层事实链分别读回；失败先区分产品缺陷、验证缺陷、工具污染、宿主边界、治理缺口和声明错位，再决定是否施工。方法强调 fresh checkout、发布物仓库外首跑、真实数据库与浏览器观察、单变量恢复、陌生读者复审、Git 分支隔离、固定 SHA 与持久载体，以及在证据足够后主动停止。案例表明，失败可以是验收系统正确工作的证据，绿灯也可能只覆盖一个窄合同。本文最后把这些做法解释为一种损失控制结构：它不能阻止 AI 漂移，不能保证零缺陷，也不能把单机三实例写成跨机高可用；它能做的是让漂移更早暴露，让错误难以直接进入主线，并为中止、回退、复算和长期审计保留路径。

关键词：对抗性工程验收；独立证据；fresh checkout；GitHub 治理；宿主边界；停止条件；AI 工程；归档

1 问题提出：完成为什么不能由作者单方面宣布

项目收尾常被理解为最后跑一遍测试、补齐文档、打一个 tag。这个流程在大多数日常维护里足够实用，但它有一个不容易看见的前提：作者知道怎样让项目成功。作者知道应该先启动哪个服务，知道哪个红灯只是环境噪声，知道 README 哪一句已经过时，也知道本机哪个目录里藏着没有写进仓库的依赖。只要这些知识没有被显式拿走，“项目完成”就可能只是“项目在作者身边能够完成”。

生成式 AI 把这个问题放大了。模型可以在同一上下文里连续完成实现、测试、解释和总结，每一步都很流畅。流畅会制造一种很强的闭合感：既然代码是按计划写的，测试也是针对这段代码写的，报告又能解释测试，那么结论似乎已经稳固。可一旦最初目标、路径假设或验收边界错了，后面的完整性反而会把错误包得更结实。

因此，本轮归档没有把目标写成“证明八个项目都好”。目标更朴素，也更难看一点：给每个项目充分暴露自己的机会；如果它只能得到 **PASS WITH BOUNDARY**、**INCONCLUSIVE**、**DEFERRED** 或 **NOT PROVEN**，就如实留下。归档不是把所有状态刷成绿色，而是让半年后的陌生人知道每个结论凭什么成立、又在哪里停止。

1.1 从庆功流程换成反证流程

测试通过 → CI 变绿 → README 写“完成” → 归档

上面这条链没有错，只是不够。测试可能只观察了自己定义的断言，CI 可能存在却没有被 required，README 可能把浏览器体验写成自动化已证明，而 workflow 根本没有启动浏览器。对抗性验收做的第一件事，是把“我有什么证据支持自己”换成“什么实验最可能推翻我”。

本轮的核心目标不是让报告更好看，而是让验收体系有资格说“不”。

1.2 方法边界

- 它不是 **Chaos Engineering** 的同义词。故障注入只是其中一部分，代码、声明、验证器、发布物、平台治理、宿主和操作者都可能是污染源。
- 它不是深度安全扫描。本轮没有把尚未审核的方法或攻击路径扫描混入主线；安全结论只能覆盖真正执行并读回的工具与规则。
- 它不是把所有仓库复制成同一套门禁。教学框架、静态文学实验、单体系统、微服务基线、规划仓和验收工具承担不同合同。
- 它不是通用正确性证明。一次验收只能在冻结对象、环境与声明范围内给出状态，不能替代未来规模、平台和时间带来的新证据。

2 材料与研究边界

案例来自一个主页仓与七个项目仓的归档收尾。八个仓库并不属于同一种产品：有教学框架、文学网页、单体业务系统、微服务基线、验证工作台，也有明确尚未实现的规划仓。正因为差异很大，统一验收不能从“每仓跑同一条命令”开始，而必须先问每个仓库公开承诺了什么。

本轮同时受到一台 16 GiB Windows 主机的资源约束。Docker Desktop、WSL、浏览器、后台应用、多套 JVM 与中间件不能被假定可以无限共存。这个限制不是需要掩盖的尴尬，而是实验条件；它要求仓库串行执行，也迫使验收者把资源停止线、残留进程和清理结果写进证据。

对象类型	代表问题	允许的归档状态
教学框架	机制是否清楚，文档是否与实现一致	PASS / DEFERRED EXPERIMENT
静态内容	权威正文、日期语义、浏览器可见事实	PASS / ACCEPTED VALIDATOR BOUNDARY
单体业务	真实角色链、MySQL 最终事实、可降级中间件	PASS / PASS WITH BOUNDARY
微服务基线	owner DB、Outbox、MQ、幂等、对账与资源前提	PASS / HOST CAPACITY BOUNDARY / DEFERRED
规划仓	愿景、非目标和路线是否诚实	PLANNED / NOT IMPLEMENTED
验收工具	能否拒绝自己的非法产物	PASS / PRODUCT FINDING / BOUNDARY

2.1 证据不是同一种东西

代码 SHA、数据库行、浏览器 Network、GitHub ruleset 和一张主页截图都可以叫“证据”，但它们证明的事实不同。固定 SHA 能指出讨论的是哪份代码，却不能证明这份代码能运行；浏览器看到页面能够说明用户界面成立，却不能证明 main 受保护；GitHub Actions 变绿可以证明 workflow 执行成功，却不能证明它是合并所必需的。

因此，本文不把证据堆成一个总分。每个结论都要写清证据对象、观察时间、适用环境与不能推出的内容。这样做会让报告里出现更多边界词，却减少了“证据很多，所以什么都证明了”的错觉。

3 先冻结，再观察

如果被验代码、测试、验收口径和宿主状态在同一轮里一起变化，失败就很难归因。一次可复算的验收至少要冻结四样东西；发现问题以后，哪一样发生改变也必须进入记录。

冻结对象	具体内容	不冻结的后果
被验对象	commit SHA、tag/Release、依赖锁	讨论对象随 main 或缓存移动
验收合同	PASS 、 FAIL 、 BOUNDARY 、 INCONCLUSIVE	看见结果后才改评分标准
验证器	脚本版本、命令、阈值、输入	规则 and 对象一起漂移，无法知道谁变了
宿主前提	内存、Docker/WSL、端口、浏览器、后台任务	环境污染被误写成产品缺陷

3.1 固定 SHA 还不够

SHA 解决身份问题，不自动解决可达问题。一个只存在于本地 reflog 的提交，作者今天可以检出，半年后可能被垃圾回收；陌生人更无法从公开 GitHub 取得它。需要长期公开复核时，身份层、公开载体和离线保存要分开设计。

层	载体	回答的问题
身份	固定 commit SHA	我们讨论的究竟是哪份代码？
公开可达	受规则约束的 tag 或 archival asset	陌生人现在能否取得该对象？
离线恢复	branch + bundle + SHA-256 + 临时 clone	远端变化后作者能否恢复完整对象图？

受保护 tag 也不是永恒保证。管理员可以改规则，仓库也可能被删除；它只能在读回时提供一个稳定、公开、不会被日常开发自然移动的锚点。把这个边界写出来，比把“immutable”当咒语更可靠。

3.2 执行账本

每次关键验收至少记录：对象 SHA、命令、开始与结束时间、退出码、宿主资源、容器 Restart/OOM、最终数据库或文件事实、产生与删除的资源、以及最后裁决。日志不是事实本身，但它能说明证据怎样被取得，也能在结论被质疑时重建调查路径。

4 九层事实链

VeriTrail 原本擅长运行事实：前后端、数据库、浏览器、中间件、失败与恢复。本轮归档又把 Repository Governance 和 Public Readability 独立出来。它们不是为了凑层级，而是因为几次现场已经证明：这些事实之间确实不能互相替代。

表 4-1 九层归档事实链

层级	范围	核心问题
L1	Source / Contract	代码、文档、状态和版本坐标是否一致
L2	Build / Test	构建、测试与静态门禁是否真实执行
L3	Runtime / Business	API、数据库、MQ 与最终业务状态是否闭合
L4	Browser / Observation	DOM、Network、Console 与可见事实是什么
L5	Failure / Recovery	故障、降级、幂等、恢复与对账是否成立
L6	Artifact / Release	公开资产、哈希与仓库外首跑是否成立
L7	Repository Governance	PR、required checks、ruleset、Pages 与 tag 如何约束变更
L8	Cleanup / Reproducibility	残留、卷、端口、临时目录与 fresh 复现是否闭合
L9	Public Presentation / Readability	陌生访客能否辨认身份、状态、边界与入口

4.1 运行事实与治理事实

GitHub Actions 全绿是执行事实：某个 workflow 在某个提交上成功了。required check 是治理事实：一个变更没有这项成功是否仍能进入主线。两者常被同一句“CI 已通过”混在一起。本轮远端读回发现，有的仓库门禁存在且通过，却未被强制；有的仓库要求 PR，却没有把 CI 作为合并条件。这不必都升级成产品 bug，但必须在归档状态里单独说清。

workflow 成功是执行事实；required check 是治理事实。

4.2 L9 不是审美附加项

如果主页首屏把规划仓写得像已完成产品，或者移动端表格让读者看不清三篇文章的关系，前八层证据仍可能被误读。L9 只问陌生访客看到什么：标题是否压在背景图文字上，入口是否可达，状态徽章是否诚实，主页与仓库首屏是否给出同一身份。它不替代产品验证，却决定证据能否被正确找到和理解。

5 失败先分类，不要先施工

失败出现时，最危险的反应是立即改代码。一次 403 可以来自后端鉴权，也可以来自合同外 Origin；一个 Catalog 被拒绝，可能是 verifier 太严，也可能是 producer 搬家以后还拿着旧地址。分类发生在修复之前，否则验收者很容易把自己的污染写成产品缺陷，甚至削弱本来正确的保护边界。

类型	识别问题	正确动作
Product defect	公开合同内的产品实现是否可重复失败	修根因，按原合同重跑正链与负链
Validation defect	验证器是否漏看事实或允许伪装	修验证范围；保留历史语义
Validation pollution	工具、缓存、地址或操作者是否改变实验	修正条件，不改产品迎合工具
Host boundary	资源、Docker、WSL、端口是否满足前提	停止、记录、单变量恢复或延期
Governance gap	检查是否真正 required，载体是否持久	独立治理施工，不冒充产品 bug
Claim / gate mismatch	公开声明是否超过门禁实际能力	收回声明或补证据，具体分析

5.1 红灯也有身份

失败不是一个统一标签。Product FAIL、Validation FAIL、Host Boundary 和 Inconclusive 的后续动作完全不同。把它们压成“测试红了”，会诱导团队降低阈值、增加 retry、mock 掉真实浏览器，或者为了让工具好用而修改产品。

第一次失败现场也不该为了报告漂亮而消失。真正有价值的记录通常是：失败时看见了什么；哪个假设被提出；只改变了什么；复验是否按预测恢复；最后数据库、浏览器或运行时的事实是什么。只有最终绿灯而没有前一段红灯，证据反而变薄了。

5.2 边界不是失败的委婉说法

16 GiB 主机跑不完一个公开承诺的完整拓扑，不等于产品错误；也不能因此写成产品通过。正确状态可以是 HOST CAPACITY BOUNDARY 或 INCONCLUSIVE，并附上下一次需要什么资源、要按什么顺序继续。边界之所以有价值，是因为它保护了实验条件，没有用 swap、降阈值或删除测试换来一枚虚假的绿色勾。

6 独立性来自信息结构

无上下文审查者未必更聪明。它的价值是没有参与施工，不知道前一个 Agent 为什么觉得自己对，也没有理由替已经写出的两千行代码辩护。独立性首先是一种信息结构，而不是一个角色名字。

阶段	输入	权限与输出
A · 发现	冻结公开对象与原始合同	只读审计，形成候选 findings
B · 施工	fresh clone + A 的有限 findings	逐条复现，只修成立的问题
C · 再审	修后的公开对象，不读 A/B 自评	主动找反例，独立裁决是否闭环

6.1 为什么不能两个施工队同时工作

本轮早期一度出现两个 Agent 在同一台机器上交叉工作：一个 fresh fixer 正在隔离目录修复，另一个长会话同时清 Docker、重启 WSL、修改 MiniSpringBoot 并准备继续安全扫描。任何一边失败，都很难判断是代码、环境还是另一边刚改过的状态。这里最先需要修的不是 Java，而是实验设计。

因此流程改成严格串行：一仓没有闭环，不切换下一仓施工；同一仓发现、修复和陌生读者复审之间用分支、fresh clone 与固定坐标隔离。这样做速度看起来慢一点，却减少了最昂贵的返工——无法证明是谁改变了事实。

6.2 陌生 finding 也要验

审查者没有上下文，不代表它天然正确。它可能误读项目边界，也可能把页面内容日期当成代码发布日期。陌生 review 的产物仍然是候选 finding，必须回到源码、公开声明和运行事实中复核。独立性降低相关错误，不授予裁判免检权。

7 单变量恢复与工具污染

当失败可能来自宿主或工具时，漂亮解释没有单变量对照有用。最小对照要求代码、测试、阈值和数据不变，只改变一个前置条件，然后观察原失败是否按预测消失。一次 A/B 只能支持假设；若启动时序、残留状态或随机初始化可能混入，应交替重复 $A \rightarrow B \rightarrow A \rightarrow B$ 。

失败现场 $\rightarrow$ 前置条件假设 $\rightarrow$ 单变量改变 $\rightarrow$ 原样复验 $\rightarrow$ 最终状态读回

7.1 浏览器控制层会骗人

DarkRoomLibrary 的登录曾连续返回 403。最初怀疑 Element Plus 输入、DOM 隐私层、验证码生命周期和 Redis；这些解释都不算荒唐，但 Network 响应最后直接给出 Invalid CORS request。验收使用的是 `http://127.0.0.1:5175`，公开合同允许的是 `http://localhost:5175`。它们都指向本机，在浏览器安全模型中却是不同 Origin。

正确 Origin 的成功与合同外 Origin 的拒绝构成正负对照，也说明 CORS 不是摆设。项目没有义务为了某个自动化控制层的习惯去放宽合同。观察工具不是上帝，观察工具本身也必须被事实反查。

7.2 Docker 的启动路径也可能成为变量

同一台 Windows 主机上，Docker Desktop 曾因 `sailor-ingest.sock` 无法移除而退出。一次从 Codex 子进程树启动失败，从 Explorer/桌面入口启动成功，只能形成 strong hypothesis，不能立刻宣布根因。宿主启动路径、残留 socket、时序与上一次退出状态都有可能参与。正确做法是记录、复现、交替对照，并在 Engine 可读之前不创建项目资源。

7.3 清理也要闭环

磁盘清理与项目归档被分成两条轨道。删除前必须确认文件内容、所有权和可恢复性；删除后还要启动相关程序、读回 Docker 卷和挂载、验证缓存能够重新取得。空间释放只能证明字节少了，不能证明软件仍然可用。一次清理若越过 ACL 保护边界，更不能只写 cleanup PASS。

8 五个代表性案例

8.1 VeriTrail：验证器拒绝自己的产物

公开 wheel 在仓库外生成 demo 时，Catalog 先在 staging 目录记录绝对 Artifact root，随后整个目录被 atomic move 到最终位置。Catalog 的内容身份、Run 和 issue 没变，物理路径绑定却过期。官方 catalog-serve 读到 ARTIFACT_ROOT_MISMATCH 后 fail-closed。

修复没有放宽 verifier，而是修 producer：让 Catalog 在最终位置建立绑定。随后用公开 wheel/sdist 重跑合法链，又把目录非法移动，确认负链仍然拒绝。这个案例最漂亮的地方不是工具永远给自己 PASS，而是它对自己的官方产物也不讲情面。

8.2 InkNarratives：门禁只能证明它真正看过的范围

静态验证曾只覆盖有限内容范围，页面可见事实和公开声明却超过门禁实际观察。陌生读者继续构造注释、script、style、template、属性名与引号内伪标签等反例，最终目标被收窄为仓库自己允许的受控 HTML 合同，而不是继续写一个通用 HTML5 parser。

这里还有一个很容易被工程语言覆盖的事实：苏轼、王维、柳永页面的 content-revised 是文学内容修订日期，不是验证器代码修改日期。工程施工发生在 8 月 28 日，并没有权利把作品日期改成 8 月 28 日。验证器负责发现变化，变化为什么发生仍要由事实解释。

8.3 DarkRoomLibrary：单体把证明路径压得很短

DarkRoomLibrary 有六身份、采购、物流、借阅、库存、Redis、RabbitMQ 和文件存储，并不算功能少。它在故障测试中表现稳定，很大原因是权威业务事实仍集中在 MySQL。Redis 与 RabbitMQ 可以降级、恢复，核心状态不需要跨多个服务各自保管一截。

这次验收同时发现一个真实密码输入边界：外部输入没有明确最大长度，validator 又连续执行多次正则扫描。修复没有把四个 CodeQL alert 当成四个独立漏洞，而是建立一个请求长度合同，再把检查改成线性字符分类。一个根因、两条边界、一次完整读回，比“消掉四个告警”更准确。

8.4 PlainJournal：微服务不是天然不稳，只是证明成本高

PlainJournal 的源码合同、14 个后端模块、436 项测试、静态质量和前端冻结安装均能重放。真正昂贵的部分在运行时：订单、库存、支付、履约、Outbox、MQ、幂等、对账和 owner database 必须重新指向同一个现实。单体靠减少变量获得可靠性，微服务靠治理变量获得可靠性。后者不是做不到，只是每拆出一个事实所有者，就多出一段必须证明的协调空间。

16 GiB 首跑中，七个核心中间件与八个业务 JVM 可以全部启动，但宿主只剩约 0.46 GiB 可用物理内存。继续硬跑会把 pagefile 和系统抖动混入业务证据，因此结论停在 INCONCLUSIVE / HOST CAPACITY BOUNDARY。32 GiB 双通道后的 Core Smoke、代表服务三实例、并发阶梯和故障恢复已经冻结成 runbook，但仍是 PLANNED / DEFERRED。

8.5 MiniSpringBoot：停止点本身也是结果

MiniSpringBoot 本来负责解释 Spring 的核心机制。归档审查中，Agent 从事务失败语义开始，继续长出连接池所有权、HTTP 资源预算、JSON 边界和 CI hardening。每一步都能讲出局部理由，整体目标却从“冻结教学项目”漂向“再造一个半工业框架”。

这段施工没有被硬做完，也没有因为已经跨 37 个文件就获得合入权。原始实验以固定 SHA、公开 archival tag 和离线 bundle 保留；其中一个独立成立的声明修正以等价 patch 单独进入教学主线。于是项目保留了三个事实：核心机制很简单，现实会逼复杂度增长，复杂度增长到越过教学目标时应该停止。

表 8-1 代表现场与最终裁决

现场	最初表象	最终裁决
VeriTrail Catalog	官方 demo 不能被官方服务读取	Product defect；修 producer，不削 verifier
InkNarratives gate	门禁绿但可见事实超出覆盖	Validation defect + accepted parser boundary
DarkRoom 403	登录失败	Validation pollution；合同外 Origin 被正确拒绝
PlainJournal 16 GiB	核心拓扑已起但 smoke 未闭合	HOST CAPACITY BOUNDARY / INCONCLUSIVE
MiniSpring hardening	实现越来越完整	Scope drift；保留实验，教学 main 不被整体污染

9 真实业务验收：最终事实由谁裁决

API 返回 200 只能证明请求被某层接受，不能自动证明库存、支付、履约和消息消费都完成。真实业务验收必须先指定事实所有者：哪个 owner database 对哪一段状

态有最终裁决权，Outbox 是否发出，consumer 是否重复执行，reconciliation 是否重新守恒。

观察层	能证明什么	不能替代什么
HTTP/API	入口接受、错误语义、响应合同	数据库最终事实与异步副作用
Owner DB	业务记录、唯一性、余额/库存	消息是否已传播和浏览器呈现
Outbox/MQ	事件发布、积压、重复与消费	上游业务提交是否真实
Browser	用户操作、DOM、Network、Console	仓库治理和后台最终对账
Reconciliation	多事实所有者最终是否守恒	未纳入对账的域外能力

9.1 三实例不是跨机高可用

代表服务三实例可以证明负载确实分散、租约只产生一次业务副作用、单实例退出后其余实例继续、重新加入与滚动过程不破坏最终事实。它不能证明跨主机、跨可用区或跨地域高可用。只要三实例仍共享同一个 Windows 宿主、Docker Engine、磁盘和网络，它们就共享大量故障域。

9.2 高并发不能只看吞吐

本轮为 PlainJournal 的 32 GiB 延期 runbook 冻结了 1、10、50、100、300、500、1000 这一组容量阶梯；这些数字是该项目在这台宿主上的实验参数，不是对抗性工程验收的普适规则。每一级同时读 HTTP 成功率与 p95/p99、JVM heap/GC、宿主物理内存、数据库连接、RocketMQ backlog、Outbox pending 与 oldest age、库存非负、幂等唯一性、重复回调副作用和最终 reconciliation。配置能够发出 1000 并发，不等于 1000 并发已经通过；其他项目应按自己的资源合同与风险面设计阶梯。

10 停止能力：避免把归档变成新一轮开发

归档阶段最难的判断常常不是“这个问题怎么修”，而是“这个问题现在应该修吗”。强 Agent 会沿着局部正确不断深入：事务语义修完发现资源边界，资源边界修完发现异常泄露，随后又补 CI。每一步都能自圆其说，最后却可能离原任务越来越远。

严格收尾 → 发现局部缺口 → 连续 hardening → 验收范围膨胀 → 新代码要求新验收

10.1 三种候选结局

裁决	适用条件	处理
保留	真实归档阻断，修复与复验链完整	进入主线或发布链
收窄后保留	根因成立，但实现扩大到域外	只保留最小合同修复
放弃 / 延期	更漂亮、更通用、未来可能有用	保留实验记录，不替代码争取合入权

已经写了很多不是证据。测试跑过、commit 存在、方案很聪明，也不自动获得进入 main 的权利。能做出来和应该合进去是两回事。把中止实验留在隔离分支，有时比把它修成一个尴尬的“加强版”更有教学价值。

10.2 停止条件应在施工前出现

- 公开合同已经满足，剩余只是理论完备或风格优化。
- 新改动会扩大 API、状态机、依赖或维护面，却没有新的归档阻断证据。
- 修复需要改变验收合同，而不是让实现重新满足原合同。
- 宿主或权限前提不足，继续只能混入新的不可归因变量。
- 陌生读者已无法在当前声明边界内构造新的有效反例。

11 16 GiB 与 32 GiB：两段证据必须同时保留

资源升级不能抹掉旧门槛。16 GiB 阶段证明的是：在当时真实后台与 Docker/WSL 条件下，PlainJournal 完整拓扑越过了可接受物理内存边界；32 GiB 阶段未来要证明的是：在新的宿主条件下，默认 Core Smoke、代表三实例、容量阶梯和故障恢复能否闭合。它们回答不同问题。

阶段	已经成立	仍未成立
16 GiB 实测	完整拓扑可启动；触及约 0.46 GiB 空闲；停止与清理完成	默认 Core Smoke、代表三实例、容量上限
32 GiB 协议	runbook、资源门槛、卷 allowlist 与执行顺序已冻结	所有运行结果仍为 PLANNED / DEFERRED
未来执行	按同一固定坐标和默认合同逐级重验	不得用新结果改写旧宿主边界

11.1 串行不是保守，而是控制变量

16 GiB 主机上，豆包、浏览器标签页、Docker Desktop、WSL 和多个 JVM 都会竞争同一物理内存。某次可用内存上升，可能只是后台应用退出；若这个变化会影响资源裁决，就必须在干净宿主重新测。纯 Git 哈希和静态合同不必因此全部重跑，但内存敏感结果不能把多变量变化写成单变量证据。

11.2 32 GiB 后的执行顺序

1. 记录空载 Windows、Docker/WSL、页文件与可用物理内存；核对固定 RocketMQ 卷和 bind-mounted 原始数据。
2. 用固定公开 SHA fresh bootstrap，逐级启动七个核心中间件，读回 secrets、schemas、Nacos configs、topics 与 buckets。
3. 按默认资源合同启动八个业务 JVM，执行交易到 owner DB、Outbox、MQ、consumer、final state 与 reconciliation 的完整链。
4. 只对代表服务做三实例，同时验证分流、租约、幂等、退出、重入和滚动过程，不冒充跨机 HA。
5. 容量按阶梯推进，每一级同时检查性能、资源和业务正确性；首次越界处就是当前机器的容量边界。
6. 容量基线稳定后再做故障恢复，最后读回数据库、Outbox、MQ、端口、进程、卷与 Git clean。

12 从验收方法到 AI 工程损失控制

这轮经历也给 FlowKernel 一类执行操作系统提供了方法输入。需要谨慎的是，证据链、GitHub 门禁和执行内核都不能保证 AI 不跑偏。模型仍可能误解目标，人在错误时也可能做出错误停止。合理的承诺只是：让漂移更难直接造成不可逆、公开或跨仓库的损失。

这里不存在一条已经校准的统一风险公式，只有几项需要分别压低的风险方向：漂移发生的可能性、Agent 的可达权限、单次操作的影响半径、发现延迟与恢复成本。

系统未必能把漂移可能性降到零，却可以缩小权限和影响半径、缩短发现延迟，并通过 Git 对象、事件账本、bundle 和清理读回降低恢复成本。这里讨论的是控制方向，不是假定这些量具有可直接相乘的统一尺度。

控制层	主要机制	能够控制	不能保证
目标与合同	冻结目标、非目标、停止线	发现局部优化越过主线	人一开始定义正确
执行隔离	branch/worktree、最小权限、资源预算	限制爆炸半径与交叉污染	Agent 不会犯错
事实验收	DB/Browser/Runtime/Release 读回	阻止声明自动升级为事实	所有域外行为正确
仓库治理	PR、required checks、ruleset、tag	阻止未批准对象自然进入主线	管理员永远不改规则
恢复与留痕	固定 SHA、bundle、日志、清理读回	支持回退、复算和追责	历史载体永不损坏

12.1 验迹、GitHub 与执行内核各守一层

验迹工作台回答行动后的世界发生了什么；GitHub 治理回答什么对象有资格进入主线和长期公开；执行内核回答 Agent 在行动时能用哪些 capability、资源和恢复路径。三者可以接起来，却不能互相冒充。一个浏览器 PASS 不会自动保护 main，一个 required check 也不会读取数据库最终事实。

12.2 只能控制损失

这套方法不是 AI 正确性保险。它的价值是：漂移发生时更早暴露，发生后更容易中止，进入主线前多一道独立裁决，已经发生的施工仍有可回退、可审计的边界。

把这个边界写进方法论很重要。若系统把“损失控制”宣传成“不会失败”，它会重新制造本文试图避免的伪证据。FlowKernel 目前仍是 Planned；这些内容只能作为设计输入，不能倒写成已实现能力。

13 一套可执行的归档协议

1. 读取仓库主页、README、设计文档、状态声明与公开 Release，先理解它是什么、不是什么。
2. 冻结代码 SHA、验收合同、验证器版本和宿主前提；把 deep security 等延期项明确排除。

3. 在干净工作区执行 L1/L2，不复用作者未声明的构建物、node_modules 或私有路径。
4. 按项目合同启动最小真实拓扑，浏览器、数据库、中间件分别读回，不让 API 200 代替最终事实。
5. 保留失败现场，先分类 Product、Validation、Pollution、Host、Governance 或 Claim mismatch。
6. 成立的产品根因进入最小施工集合；治理、宿主和工具问题进入各自轨道。
7. 修复后执行正向、负向、恢复和清理；必要时采用 $A \rightarrow B \rightarrow A \rightarrow B$ 排除时序与残留。
8. 让无上下文读者只看修后的公开对象，允许它推翻施工者结论；finding 本身也必须被复核。
9. 通过 PR、required checks、ruleset、Release/tag 和远端读回完成 L7；用首页与仓库首屏完成 L9。
10. 给每个仓库写出 PASS、BOUNDARY、DEFERRED、NOT PROVEN 等最终身份，说明为什么现在应该停。

13.1 施工门槛

一个 finding 进入施工集合前，至少要回答：它是否在公开合同内可重复；根因证据是否比表象更硬；修复是否会改变核心机制或扩大 API；能否用原合同做正负复验；若不修，半年冻结会造成什么实际后果。答不清时，正确动作通常是继续审查或记录边界，不是先写代码。

13.2 完成定义

- 仓库身份、状态、版本坐标和公开入口彼此一致。
- 公开最小路径能够在 fresh checkout 或发布物环境中真实执行，或被明确标记为资源边界。
- 业务最终事实、浏览器事实、治理事实和清理事实分别有自己的读回。
- 失败、延期与 NOT PROVEN 没有被绿色摘要抹掉。
- 一个不知道施工历史的人，仍能找到证据、理解边界，并知道下一步从哪里继续。

14 局限与反例

第一，本文仍是一名作者、有限仓库和一台宿主机上的纵向复盘，不是大样本研究。陌生读者降低了叙事继承，但多个模型仍可能共享训练偏差；人类裁决也可能错。

第二，证据工具本身可能污染事实。浏览器控制层会误填，截图会截断，Docker 启动链会留下状态，数据库查询也可能读错 schema。所谓独立证据只是降低相关性，并不自动获得绝对真理。

第三，本轮没有完成 Codex deep-security 或 attack-path scan。DarkRoomLibrary 与 PlainJournal 的 CodeQL 公开告警已按当时规则读回为零，不代表六个未配置分析的仓库“零安全问题”，也不代表攻击面已穷尽。

第四，16 GiB 与同宿主三实例无法替代跨机、跨可用区、长期高频并发和真实社区环境。未来 32 GiB 结果也只会扩大已观察范围，不会把单机事实升级成互联网规模。

第五，归档不是永远冻结。依赖、平台规则、浏览器与操作系统都会变化。本文给出的只是 2026 年 8 月固定坐标上的可复算结论；半年后继续开发时，应从这些坐标重新建立新一轮事实链。

15 结论

本轮收尾最重要的收获，不是又找到几个 bug，而是验收本身也接受了验收。门禁绿了可以继续问它究竟证明什么；测试红了可以先判断项目是否正确拒绝了不满足条件的环境；AI 写出了更严谨的实现，也可以因为越过目标而不合入。

归档意义上的“完成”不是代码零缺陷。它更接近一种一致：代码、测试、运行事实、发布物、文档叙述、Git 历史、远端治理、失败记录和边界声明基本指向同一个现实。这个现实里允许有 PASS WITH BOUNDARY、DEFERRED 和 NOT PROVEN，只是不允许它们被写成别的东西。

对抗性工程验收最终保护的并不是绿色勾，而是停止权。项目不是做不动了才停，也不是因为 AI 已经写了很多就继续；当公开合同、独立证据、治理与清理已经闭合，剩余工作只会扩大风险时，系统能够说：现在有足够理由停在这里。

真正的归档句号不是“终于全绿了”，而是“现在我们知道，为什么可以停”。

致谢（不严肃）

感谢所有看起来很合理、最后却被 Network、数据库、Git 对象或宿主内存推翻的解释。它们提醒作者：观察工具会骗人，操作者也会污染实验；一个无上下文读者未必更聪明，只是不知道前一个模型为什么觉得自己对。也感谢那台 16 GiB 的小电脑，它没有提供无限资源，却逼迫每次启动、停止、残留和边界都留下了名字。

附录 A 状态词典

状态	含义	禁止的解释
PASS	冻结合同内的证据链闭合	域外能力也正确
PASS WITH BOUNDARY	核心结论成立，边界仍需显式携带	等同完全通过
REFUTED	已有反例或事实冲突	用新叙事覆盖失败
INCONCLUSIVE	当前证据不能区分候选解释	基本通过或基本失败
HOST CAPACITY BOUNDARY	宿主资源前提不足，实验停止	产品缺陷或产品通过
PLANNED / DEFERRED	协议已定义，执行尚未发生	已经验证
NOT PROVEN	没有足够证据声明能力	能力不存在
NOT APPLICABLE	该验收域不属于当前合同	验证失败

附录 B 最小证据记录

字段	建议内容
object_identity	repo、commit SHA、tag/Release、asset SHA-256
contract	PASS/FAIL/BOUNDARY/INCONCLUSIVE 与停止线
environment	OS、内存、Docker/WSL、浏览器、后台任务、端口
execution	命令、开始/结束、退出码、关键输出、超时
observation	DB、Browser、Runtime、GitHub API 与文件哈希
perturbation	只改变了什么，哪些变量保持不变
cleanup	进程、端口、卷、临时目录、工作树最终状态
verdict	结论、适用边界、延期项、下一次固定入口

附录 C 本轮固定案例坐标

项目	固定坐标与边界
VeriTrail	f50d5e1abfc8fc052a36a5be1d5e09047625ebbf
InkNarratives	56ad434c66bc3aa7d8835823b39ffb5720c93c2b
DarkRoomLibrary	b83ba4ed858141e4e66afbea7d6ddff642fb5bd8
PlainJournal	37009ea78ecf268cf9462f1499b5b6717faa8821 ; 32 GiB 协议 仍 为 PLANNED / DEFERRED
MiniSpringBoot	教 学 基 线 946725ed80fed4a364f86d65e4b98962322345e7 ; Raw Hardening 870c890; 另有 archival tag 与 bundle

这些坐标用于说明本文讨论的对象，不把仓库平台、规则或宿主状态冻结成永恒事实。远端治理与机器环境必须携带读回时间重新判断。